The HDC Classifier — A Complete Guide

A from-scratch explanation of the Hyperdimensional Computing (HDC) classifier built in this project: the theory, the math, the encoding pipeline, the hardware mapping, and a fully worked example.

---

1. The big picture

This project builds a **classifier** — it reads a short window of a biosignal (4-channel EMG from the forearm) and decides **which hand gesture** is being performed (e.g. closed hand, open hand, pinch, point, rest).

Instead of a neural network, it uses **Hyperdimensional Computing (HDC)**, also called *Vector Symbolic Architectures*. The core idea:

> Represent everything — sensor channels, signal values, time steps, and whole gestures — as very long binary vectors (here **1024 bits**), and do all "thinking" with three cheap bitwise operations: **bind**, **bundle**, and **permute**, plus a **similarity** check.

Why anyone would do this:

- **Cheap hardware.** The operations are XOR, popcount, and majority voting — no multipliers, no floating point. Perfect for an FPGA / the Zynq fabric.
- **One-/few-shot learning.** A class is learned by *averaging* example vectors, not by gradient descent over many epochs.
- **Robustness.** Information is spread across all 1024 bits, so flipping a few bits (noise, a pruned bit, a hardware fault) barely changes the answer.

---

2. Why *hyper*dimensional? The blessing of dimensionality

Pick two **random** 1024-bit vectors. On average they differ in exactly **half** their bits (512), because each bit is an independent coin flip. The number of differing bits (the **Hamming distance**) follows a binomial distribution:

```
mean differing bits = D / 2 = 512
std dev             = sqrt(D) / 2 = 16   (for D = 1024)
```

So two *unrelated* vectors are almost always ~50 % apart, and it is *astronomically unlikely* for them to be, say, 30 % apart by chance. This is called **quasi-orthogonality**: in high dimensions, random vectors are "nearly perpendicular" and therefore act like distinct, non-interfering symbols.

That single fact is what makes HDC work:

- Different concepts get random vectors → they don't collide.
- Related concepts get *deliberately* similar vectors → similarity is meaningful.

- We can pile many vectors together and still pull them apart, because noise averages out across 1024 dimensions.

The bigger `D` is, the more reliable this separation — which is exactly the **dimension knob** the project studies (we found D=1024 already saturates spatial accuracy; see §11).

---

3. The representation: binary hypervectors (BSC)

This project uses the **Binary Spatter Code (BSC)** model:

- A hypervector is **1024 bits**, each `0` or `1`.
- "Random" vectors have ~512 ones and ~512 zeros (balanced).
- Similarity between two vectors `A`, `B` is measured by **Hamming distance** `dist(A,B)` = number of bit positions where they differ. Small distance = similar; ~512 = unrelated; ~1024 = opposite.

A convenient equivalent: map bits `0→−1`, `1→+1`; then similarity is a dot product. Hardware uses the bit form (XOR + popcount); the math is the same.

---

4. The three operations (with toy examples)

To keep examples readable we use **D = 8** bits. The real system uses 1024.

4.1 Bind — XOR (combine "what" with "which")

Bind ties two hypervectors together into a new one that is *dissimilar to both inputs* but *reproducible*. In BSC, bind is bitwise **XOR** ($\oplus$).

```
A    = 1 0 1 1 0 0 1 0
B    = 0 0 1 0 1 1 1 0
A⊕B  = 1 0 0 1 1 1 0 0
```

Key properties:

- **Invertible:** `(A⊕B)⊕B = A`. You can "unbind".
- **Similarity-preserving:** `dist(A⊕K, B⊕K) = dist(A, B)` — binding both sides with the same key `K` is just a fixed bit-shuffle, so distances are unchanged.
- **Result looks random vs inputs:** `A⊕B` is ~half-distance from both `A` and `B`. So a bound pair is a brand-new symbol.

We use bind to attach a **value** to the **channel it came from**: `channel_id ⊕ value_vector`.

4.2 Bundle — majority (superpose "this AND that")

Bundle merges several hypervectors into one that is *similar to all of them* — a set or "average". In BSC, bundle is **bitwise majority vote** (per bit position, output the value that appears most often).

```
V1 = 1 0 1 1 0 0 1 0
V2 = 1 1 1 0 0 1 1 0
V3 = 0 0 1 0 1 1 1 1
sum= 2 1 3 1 1 2 3 1     (count of 1s per column, out of 3)
maj= 1 0 1 0 0 1 1 0     (1 if count > 3/2)
```

Key properties:

- The result is **close to every input** (it "remembers" all of them).
- **Capacity is limited:** bundle too many and they blur together — which is why the *count* of accumulated vectors matters (the hardware **counter width** `CNT_W` is a design knob).
- **Ties** (even number of inputs, 50/50 split) need a rule. This project breaks ties to **0** (matches the RTL `bundle_unit` threshold `count > n/2`).

We use bundle to (a) fuse the 4 channels into one record, and (b) average many training examples into one class prototype.

4.3 Permute — cyclic shift (encode order / position)

Permute (`ρ`) rotates a hypervector's bits by one position. It produces a vector dissimilar to the original, used to mark **position in a sequence**.

```
R       = 1 0 1 1 0 0 1 0
ρ(R)    = 0 1 0 1 1 0 0 1     (rotate right by 1)
ρ²(R)   = 1 0 1 0 1 1 0 0     (rotate by 2)
```

Key properties:

- **Reversible** and **similarity-preserving** (it's just a fixed reordering).
- `ρ^t(R)` is quasi-orthogonal to `R` for `t ≠ 0`, so "the value at time *t*" is distinguishable from "the value at time 0". This is how we encode **temporal order** in an n-gram (§6.2).

4.4 Similarity — Hamming distance / popcount

To compare a query vector against stored prototypes we count differing bits:

```
dist(A,B) = popcount(A XOR B)
```

`popcount` (number of set bits) is a single, cheap hardware primitive. The class whose prototype has the **smallest** Hamming distance wins.

---

5. Memories: turning real-world quantities into vectors

5.1 Item Memory (iM) — a codebook of random symbols

For each discrete **identity** that has no inherent ordering — here the **4 EMG channels** — we draw one fixed random hypervector. These are mutually quasi-orthogonal, so "channel 1" and "channel 3" never get confused.

5.2 Continuous Item Memory (CiM) — graded level vectors

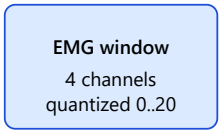
Sensor **values** *do* have an ordering (level 5 is closer to 6 than to 20). We want **similar values to get similar vectors**. So we build level vectors by starting from a random vector for level 0 and **flipping a fixed chunk of bits at each step**:

```
CiM(0)  = random
CiM(L)  = CiM(L−1) with the next D/(2·Lmax) bits flipped
```

Result: `dist(CiM(a), CiM(b))` grows smoothly with `|a − b|`. Adjacent levels are nearly identical; the extremes (level 0 vs level 20) are ~half-distance apart. This is how an analog amplitude becomes an HDC-friendly vector.

---

6. The encoder: from EMG window to one hypervector



per channel c iM[c] XOR CiM(v_c) (bind: value to channel) majority over 4 ch = record R[t]

temporal n-gram G = majority over t of ρ^t(R[t]) (permute marks time)

1024-bit gesture vector (the "query")

6.1 Spatial record — one time step

At each sample `t`, bind every channel's value to that channel's identity, then bundle the four together:

```
R[t] = majority over channels c of ( iM[c] XOR CiM(value_c[t]) )
```

`R[t]` is a single 1024-bit vector that says "*these four channels had these four values at this instant*".

6.2 Temporal n-gram — capturing motion

A gesture is a *trajectory*, not a snapshot. We combine `N` consecutive records, using **permute** to stamp each with its position in time, then bundle:

```
G = majority over t = 0..N−1 of  ρ^t( R[t] )
```

Because `ρ^0, ρ^1, …` are quasi-orthogonal, the order is preserved: "value rising then falling" encodes differently from "falling then rising". The best `N` differs per subject (3–5 in this dataset).

The output `G` is the **query hypervector** for the whole window.

---

7. Training: building the Associative Memory (AM)

Learning a class is just **bundling all its training examples**:

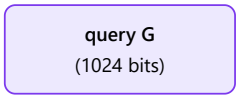
```
prototype[k] = majority over all training G belonging to class k
```

Each class prototype is one 1024-bit vector that is *similar to every example of that gesture and dissimilar to the others*. The set of prototypes (one per gesture) is the **Associative Memory (AM)**.

That's it — no backprop, no epochs. Add more examples by bundling them in. This is why HDC is attractive for **on-device, few-shot** learning.

---

8. Inference: nearest-prototype classification



prototype[1] (closed) prototype[2] (open) prototype[3] (pinch) prototype[k] ...

popcount(G XOR pk) Hamming distance to each prototype

argmin = predicted

To classify a new window: encode it to a query `G`, compute the Hamming distance to every class prototype, and pick the **closest** one.

```
predicted_class = argmin over k of  popcount( G XOR prototype[k] )
```

Optionally a **pruning mask** (a fixed 1024-bit pattern) is ANDed in first, so only the most useful bit positions are compared — saving hardware with little or no accuracy loss (this is one of the project's research knobs).

---

9. A complete worked micro-example (D = 8)

Two channels, values quantized to a few levels, N = 1 (spatial only), to show the full path end to end.

```
Item memory (random channel IDs):
  iM[1] = 1 0 1 1 0 0 1 0
  iM[2] = 0 1 1 0 1 0 0 1

Level vectors (CiM, similar levels -> similar vectors):
  CiM(0) = 0 0 0 0 1 1 1 1
  CiM(1) = 1 0 0 0 1 1 1 0   (2 bits flipped vs level 0)

Encode a sample with channel1=level1, channel2=level0:
  b1 = iM[1] XOR CiM(1) = 1 0 1 1 0 0 1 0 XOR 1 0 0 0 1 1 1 0 = 0 0 1 1 1 1 0 0
  b2 = iM[2] XOR CiM(0) = 0 1 1 0 1 0 0 1 XOR 0 0 0 0 1 1 1 1 = 0 1 1 0 0 1 1 0
  R  = majority(b1, b2) with tie->0:
       col sums = 0 1 2 1 1 2 1 0  (out of 2);  >1 ? -> 0 0 1 0 0 1 0 0

Suppose training gave these class prototypes:
  P[A] = 0 0 1 0 0 1 0 1
  P[B] = 1 1 0 0 1 0 1 0

Classify R = 0 0 1 0 0 1 0 0:
  dist(R, P[A]) = popcount(R XOR P[A]) = popcount(0 0 0 0 0 0 0 1) = 1
  dist(R, P[B]) = popcount(R XOR P[B]) = popcount(1 1 1 0 1 1 1 0) = 6
  -> nearest is P[A]  =>  predicted class = A
```

Everything above is integer/bitwise — exactly what the RTL does, just at 1024 bits instead of 8.

10. Why this is great for FPGA / Zynq — and how it maps to the RTL

HDC step	Operation	Hardware	RTL module
Bind	bitwise XOR	XOR gates (1 cycle, fully parallel)	`xor_permute_top.sv`
Permute	cyclic shift	wires / rotate	`permute_stage.sv`
Value/ID lookup	table read	BRAM/ROM	`item_memory.sv` (was `simple_bind_rom.sv`)
Bundle	majority vote	per-bit counters + threshold	`bundle_unit.sv`
Pruning	per-bit AND	mask register	`pruning_mask.sv`
Similarity	XOR + popcount + argmin	popcount tree + comparator	`popcount_am.sv`
Host interface	register/stream	AXI4-Lite / AXI-Stream	`hdc_axi_lite_wrapper.sv`, `hdc_stream_wrapper.sv`

No multipliers, no floating point — just XOR, counters, popcount. The whole 1024-bit datapath is bit-parallel, so one hypervector operation is a handful of clock cycles. The PS (ARM core) streams EMG windows

in and reads the predicted gesture out.

11. This project's concrete settings and results

- **Dimension D = 1024** (the headline design point; the core is parameterised so D can be swept).
- **4 channels, 5 gestures**, enveloped EMG quantized to **21 levels**.
- **Bind = XOR, permute = cyclic shift, bundle = majority (tie→0), similarity = Hamming/popcount.**
- **Training = majority-bundled class prototypes** (one-shot per class, then averaged).

Measured baseline (reproduced from Rahimi et al., ICRC 2016, on their data):

Model	Spatial	Spatiotemporal
This project (BSC, D=1024)	**90.30 % ± 0.13**	89.25 % ± 0.89
Bipolar MAP anchor (D=10000)	90.36 %	96.04 %
Original paper	90.8 %	97.8 %

The binary 1024-bit model matches the much larger bipolar model on the spatial task — strong evidence the small, hardware-friendly design is sound.

12. The design knobs (what the research explores)

HDC exposes a few dials that trade accuracy for area/energy — the heart of the project's novelty:

- **D (dimension):** fewer bits → smaller, faster, less energy. We found spatial accuracy is nearly flat from D=256 to 10000 → big savings possible.
- **CNT_W (bundle counter precision):** how finely the majority counters track the vote. Lower precision saves flip-flops.
- **Bit-position pruning:** keep only the most discriminative bit positions (informed) vs a random subset. *Informed* > *random* is a key claim.
- **Cross-subject mask transfer:** can a pruning mask learned on one person work on another? (Reuse / generalisation study.)

13. Glossary

Term	Meaning
Hypervector	A very long vector (here 1024 bits) used to represent a concept.
HDC / VSA	Hyperdimensional Computing / Vector Symbolic Architecture.
BSC	Binary Spatter Code — the binary `{0,1}` HDC model used here.

Term	Meaning
Bind (XOR)	Combine two vectors into a new, dissimilar, reversible one.
Bundle (majority)	Superpose vectors into one similar to all (a set/average).
Permute (ρ)	Cyclic bit-shift; encodes order/position.
Hamming distance	Number of differing bits; the similarity measure.
popcount	Count of `1` bits — the hardware similarity primitive.
Item Memory (iM)	Codebook of random vectors for unordered IDs (channels).
Continuous IM (CiM)	Level vectors with graded similarity for ordered values.
Record	One-time-step vector fusing all channels.
n-gram	Permute+bundle of N records → temporal/gesture vector.
Associative Memory (AM)	The set of per-class prototype vectors.
Prototype	A class's representative vector (bundle of its examples).
Quasi-orthogonality	Random hi-D vectors are ~50 % apart → act independent.

14. Further reading

- P. Kanerva, "*Hyperdimensional Computing: An Introduction…*", Cognitive Computation, 2009 — the foundational tutorial.
- A. Rahimi et al., "*Hyperdimensional Biosignal Processing: A Case Study for EMG-based Hand Gesture Recognition,*" IEEE ICRC 2016 — the anchor paper this project reproduces and builds on.